

POP para Extração e Análise de Dados em Sistemas de Informação via Banco de Dados MySQL: no Contexto de Atuação do Engenheiro de Produção

Gustavo Gonçalves Barcelos (Universidade Federal de Goiás - UFG)

João Vitor Bueno dos Reis (Universidade Federal de Goiás - UFG)



Este artigo apresenta um Procedimento Operacional Padrão (POP) para acesso, extração e análise de dados diretamente em banco de dados relacional MySQL no contexto da Engenharia de Produção. A motivação é a necessidade de autonomia analítica do engenheiro de produção, que historicamente recebe dados filtrados, atrasados e agregados por camadas intermediárias de TI. O trabalho fundamenta-se nos conceitos de Sistemas de Informação Gerencial (SIG), arquitetura de software em camadas e modelagem de bancos de dados relacionais, articulando o ciclo dado, informação, conhecimento com a prática de consultas SQL. São abordados a evolução histórica dos bancos de dados, a modelagem conceitual, normalização de dados, tipos de relacionamento, além das diferenças arquiteturais e operacionais entre bancos relacionais e NoSQL. O procedimento documentado descreve as etapas de conexão ao MySQL via Python, escrita de queries e geração de indicadores operacionais, usando como contexto ilustrativo uma fábrica de componentes automotivos. Os resultados

esperados compreendem a capacitação do engenheiro para responder perguntas analíticas com dados primários e atualizados, reduzindo a dependência de relatórios prontos.

Palavras-chave: Banco de dados relacional; MySQL; SQL; Engenharia de Produção; SIG; procedimento operacional padrão; análise de dados.

1. Introdução

A transformação digital das organizações produtivas tem colocado o dado como um dos principais ativos estratégicos da empresa moderna. Schwab (2016), ao caracterizar a Quarta Revolução Industrial, destaca que a convergência entre sistemas físicos e digitais, por meio de tecnologias como sistemas ERP, MES e WMS, computação em nuvem e inteligência artificial, gera volumes sem precedentes de dados operacionais, exigindo das organizações novas capacidades analíticas para transformar esses dados em decisões. Nesse cenário, a gestão eficaz da informação deixa de ser uma função exclusiva da área de Tecnologia da Informação e passa a ser uma competência essencial de todas as áreas funcionais da empresa, incluindo a Engenharia de Produção. Laudon e Laudon (2020) definem um Sistema de Informação como um conjunto de componentes inter-relacionados que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisão e o controle em uma organização. Os autores ressaltam, contudo, que a maioria dos profissionais acessa essa informação por camadas intermediárias, relatórios pré-formatados, planilhas exportadas e dashboards com escopo predefinido, o que implica perda de granularidade, atraso temporal e viés de seleção sobre os dados apresentados. Davenport e Harris (2007), em seu estudo sobre organizações orientadas por análise (analytics-driven), demonstram que as empresas que constroem vantagem competitiva a partir de dados são precisamente aquelas cujos profissionais operacionais têm acesso direto às fontes primárias de informação, sem depender exclusivamente de relatórios gerados por terceiros.

No âmbito da Engenharia de Produção, essa limitação é particularmente crítica. Slack, Brandon-Jones e Johnston (2018) argumentam que a gestão de operações exige monitoramento contínuo de indicadores de desempenho, como eficiência de linha, tempo de ciclo, nível de estoque e taxa de defeitos, e que a qualidade dessas decisões depende diretamente da atualidade e da precisão dos dados disponíveis. Quando o Engenheiro de Produção depende de relatórios elaborados por outra área, o dado chega filtrado, atrasado e agregado demais para sustentar análises de causa raiz ou simulações de cenário. Corrêa e Corrêa (2017) reforçam essa perspectiva ao apontar que a integração entre sistemas de informação e decisões operacionais é um dos pilares da gestão moderna da produção, e que a capacidade de interpretar dados diretamente dos sistemas transacionais amplia significativamente a autonomia do gestor de produção.

A formalização de procedimentos de acesso a dados como Procedimentos Operacionais Padrão (POP) é respaldada pela literatura de qualidade e gestão de processos. A ABNT NBR ISO 9001:2015 estabelece que a organização deve manter informação documentada na extensão necessária para apoiar a operação de seus processos e reter informação documentada para ter confiança de que os processos foram realizados conforme planejado (ABNT, 2015). Nesse sentido, um POP que descreve sistematicamente o procedimento de extração e análise de dados em bancos de dados atende a dois requisitos simultâneos: garante a repetibilidade e a rastreabilidade do processo analítico, e reduz a variabilidade nos resultados gerados por diferentes analistas.

Do ponto de vista técnico, Date (2004) define um banco de dados como um repositório de dados persistentes, utilizado pelos sistemas de aplicação de uma determinada empresa, e destaca que o modelo relacional, proposto originalmente por Codd (1970), oferece independência de dados, integridade referencial e uma linguagem declarativa (SQL) que permite ao usuário final expressar o que deseja obter sem precisar especificar como o sistema deve obtê-lo. Essa característica torna o SQL uma ferramenta acessível a profissionais não especializados em programação, desde que devidamente capacitados, o que inclui o Engenheiro de Produção que necessita realizar análises operacionais de forma autônoma.

Diante desse contexto, este trabalho justifica-se pela necessidade de formalizar o processo de acesso direto a bancos de dados como uma prática profissional estruturada e replicável no âmbito da Engenharia de Produção. A questão central que orienta a pesquisa é: de que forma um Engenheiro de Produção pode estruturar um Procedimento Operacional Padrão para extração e análise de dados operacionais diretamente em um banco de dados relacional?

O objetivo geral é elaborar um POP, no formato de artigo científico segundo as normas do ENEGEP (ABEPRO), que descreva o procedimento de acesso, extração e análise de dados em um banco de dados real, aplicado ao contexto da Engenharia de Produção. Os objetivos específicos são: (i) Revisar os fundamentos teóricos de Sistemas de Informação Gerencial, a evolução histórica dos bancos de dados, padrões de arquitetura de software e tipologias de SGBD; (ii) Apresentar as abordagens estruturais de modelagem, englobando diagramas conceituais, normalização de dados e relações de agregação/composição; (iii) Documentar detalhadamente o procedimento técnico de conexão scriptada e as queries utilizadas para extração de dados; (iv) Demonstrar a geração de indicadores de desempenho operacional a partir das fontes de dados primárias tratadas.

2. Fundamentação Teórica

2.1 Sistemas de Informação Gerencial (SIG) e lógica analítica

Os Sistemas de Informação Gerencial (SIG) são responsáveis por transformar dados operacionais em informações úteis para apoio à tomada de decisão. Esses sistemas integram dados provenientes de diferentes áreas da organização, como produção, estoque e qualidade, permitindo análise estruturada dos processos (LAUDON; LAUDON, 2020).

No contexto da Engenharia de Produção, os SIG estão diretamente associados a sistemas como ERP, MES e WMS, que registram continuamente eventos do chão de fábrica. Entretanto, o valor desses sistemas não está apenas no armazenamento dos dados, mas na capacidade de transformá-los em conhecimento aplicável.

Entretanto, o valor desses sistemas não está apenas no armazenamento dos dados, mas na capacidade de transformá-los em conhecimento aplicável. Esse processo analítico sequencial é compreendido pelo modelo clássico adaptado de Laudon e Laudon (2020), detalhado no Quadro 1.

Quadro 1 — Transformação de dados em conhecimento

Nível	Descrição	Exemplo em EP
--------------	------------------	----------------------

Dado	Registro bruto, sem contexto	“150 unidades em estoque”
Informação	Dado interpretado em contexto	“Estoque abaixo do mínimo”
Conhecimento	Informação aplicada à decisão	“Geração automática de ordem de reposição imediata”

Fonte: Adaptado de Laudon e Laudon (2020).

Assim, observa-se que o banco de dados atua como base para a retenção persistente do dado, enquanto as manipulações lógico-declarativas operam a transformação informacional para fundamentar o conhecimento corporativo.

2.2 Evolução Histórica dos Bancos de Dados

A evolução dos repositórios de dados reflete o imperativo de adaptação às demandas de escala, flexibilidade e performance informacional das organizações ao longo das últimas décadas.

Iniciando-se pela década de 1960 com o Modelo Hierárquico e o IMS: O marco inicial dos SGBDs corporativos consolida-se com o surgimento do Information Management System (IMS) pela IBM para dar suporte ao programa espacial Apollo. O IMS organizava dados sob uma topologia hierárquica em árvore. Embora demonstrasse excelente desempenho em operações predefinidas e volumosas, a rigidez estrutural impunha severas limitações para a realização de consultas dinâmicas não planejadas no projeto original.

Em diante, a década de 1970 através do Modelo Relacional de Codd: Edgar F. Codd (1970) publicou o artigo seminal que revolucionou a computação ao propor a estruturação matemática dos dados em tabelas (relações), dissociando a representação lógica do armazenamento físico. O modelo relacional introduziu garantias de consistência via chaves primárias e estrangeiras, trazendo flexibilidade e integridade às transações organizacionais.

A partir das décadas de 1980 e 1990 — A Consolidação do SQL e SGBDs Comerciais: A linguagem SQL (Structured Query Language) foi padronizada internacionalmente, tornando-se a interface universal para manipulação de dados relacionais. SGBDs comerciais e de código aberto como Oracle, SQL Server, MySQL e PostgreSQL expandiram-se nas corporações industriais como o cerne de sistemas integrados de gestão (ERPs), pautando-se sob a confiabilidade transacional estrita. Diante da década de 2000, com o Movimento NoSQL: O advento da Web 2.0, o crescimento exponencial do e-commerce e a emergência do

Big Data evidenciaram os gargalos de escalabilidade vertical dos bancos relacionais para dados volumosos semiestruturados. Surgem os bancos NoSQL (Not Only SQL), categorizados em orientados a documentos, chave-valor, colunares e grafos, priorizando a escalabilidade horizontal distribuída.

Por fim, a década de 2010 até o presente Foi e é marcada pela Polyglot Persistence (Persistência Poliglota): Formalizado por Sadalage e Fowler (2012), o paradigma atual assume que nenhum SGBD isolado atende otimizada a todas as demandas de uma infraestrutura moderna. Sistemas industriais complexos passam a empregar múltiplos tipos de bancos de dados concomitantemente, alocando cada tecnologia para o padrão de acesso onde ela oferece sua máxima eficiência.

2.3. Arquitetura de Software e o Papel do Banco de Dados

2.3.1 Arquitetura em Camadas (N-Tier)

A arquitetura em camadas (N-Tier) é um dos padrões mais consolidados no desenvolvimento de sistemas corporativos. Seu princípio fundamental é a separação de responsabilidades: cada camada possui uma função específica e comunica-se apenas com as camadas adjacentes, reduzindo o acoplamento e facilitando a manutenção e a evolução do sistema (PRESSMAN; MAXIM, 2020). No modelo de três camadas clássico, encontrado em ERPs e sistemas MES industriais, a organização é a seguinte: (i) camada de apresentação, responsável pela interface com o usuário, dashboards, relatórios, telas de apontamento; (ii) camada de aplicação ou negócio, onde residem as regras da organização, cálculo de OEE, sequenciamento de ordens, validações de qualidade; e (iii) camada de dados, responsável pelo armazenamento persistente e recuperação das informações, o banco de dados.

Essa separação tem implicações diretas para o engenheiro de produção: os relatórios e dashboards que ele consulta diariamente são produtos da camada de apresentação, que recebe dados já filtrados e agregados pelas camadas superiores. Acessar o banco diretamente significa bypassar essas camadas intermediárias e trabalhar com os dados em seu estado mais bruto e completo.

2.3.2 Arquitetura Padrão MVC (Model-View-Controller)

Sistematizado originalmente por Trygve Reenskaug em 1979, o padrão arquitetural MVC é direcionado para a estruturação lógica de interfaces de software, decompondo os fluxos internos de interação em três componentes principais.

O primeiro sendo o Model (Modelo): Componente nuclear responsável pela gestão direta do

estado dos dados, encapsulando as regras de validação estrutural e a comunicação com as tabelas ou coleções de persistência física do banco de dados.

Em seguida, o View (Visão): A representação visual e interface gráfica apresentada ao usuário final. Ela exibe as informações disponibilizadas pelo componente Model sem deter qualquer gerência sobre as regras que regem esses dados.

E no final, o Controller (Controlador): O intermediário lógico de controle. Ele intercepta as requisições, eventos e ações executadas pelo usuário na View, processa e aciona os métodos necessários no Model, redefinindo a interface que deve ser renderizada em resposta. Enquanto a arquitetura N-Tier foca na segregação macroscópica de infraestrutura física e lógica corporativa (Servidor de Banco → Servidor de Aplicação → Cliente), o MVC opera prioritariamente dentro do escopo do design interno de uma aplicação para desemaranhar o código de controle da interface visível do usuário.

2.3.3 Arquitetura Hexagonal (Ports and Adapters)

Proposta por Alistair Cockburn em 2005, a arquitetura hexagonal (também chamada Ports and Adapters) representa uma evolução em relação ao MVC tradicional. Seu princípio central é proteger o núcleo de negócio da aplicação de dependências externas, como banco de dados, interfaces de usuário e serviços de mensageria, por meio de portas (interfaces abstratas) e adaptadores (implementações concretas) (COCKBURN, 2005).

Nesse modelo, o banco de dados é explicitamente tratado como um detalhe de infraestrutura: o núcleo da aplicação define uma interface de repositório (por exemplo, `OrdemProducaoRepository`), e o adaptador concreto implementa essa interface para o MySQL, PostgreSQL ou MongoDB. Isso significa que, do ponto de vista arquitetural, trocar o SGBD não exige reescrever a lógica de negócio, apenas substituir o adaptador. Para o engenheiro de produção, isso reforça que o banco de dados é uma peça substituível e que o conhecimento de SQL transcende qualquer SGBD específico.

2.3.4 Clean Architecture e o Princípio da Inversão de Dependência

A Clean Architecture (Martin, 2017) estende o desacoplamento organizando o ecossistema de software em anéis concêntricos de responsabilidade lógica. A premissa central estabelece que as linhas de dependência de código devem apontar unidirecionalmente para os círculos internos. Consequentemente, as entidades fundamentais e casos de uso de produção ocupam o núcleo central intocável, ao passo que os frameworks, interfaces de usuário e SGBDs orbitam na periferia extrema como extensões plugáveis e facilmente substituíveis.

O Quadro 2 consolida o posicionamento estrutural conferido ao banco de dados sob o escopo destas principais matrizes arquiteturas.

Quadro 2 — Principais Padrões Arquiteturais e o Papel do Banco de Dados

Padrão	Origem	Papel do BD	Aplicação em EP
N-Tier (camadas)	Anos 1990	Camada de dados isolada	ERP, MES, WMS corporativos
MVC	Reenskaug (1979)	Model — acessa e persiste dados	Sistemas web de gestão de produção
Hexagonal	Cockburn (2005)	Adaptador de infraestrutura substituível	Sistemas com múltiplos SGBDs
Clean Architecture	Martin (2017)	Plugin periférico, detalhe de implementação	Sistemas de alto desacoplamento

Fonte: elaborado pelos autores com base em Fowler (2002) e Martin (2017).

2.4 Bancos de dados e organização da informação

2.4.1 Modelo relacional

O modelo relacional, proposto por Codd (1970), organiza os dados em tabelas compostas por linhas (registros) e colunas (atributos). Cada tabela possui uma chave primária responsável por identificar unicamente cada registro, enquanto os relacionamentos entre tabelas são realizados por meio de chaves estrangeiras (ELMASRI; NAVATHE, 2011).

Esse modelo é amplamente utilizado em sistemas industriais devido à sua consistência e facilidade de consulta por meio da linguagem SQL.

2.4.2 Modelagem de Dados: Agregação versus Composição Estrutural (UML)

No mapeamento conceitual da estrutura da fábrica por diagramas de classes UML, faz-se imperativo discernir a semântica das relações que acoplam objetos operacionais.

Inicialmente, a agregação Estrutural (Relação Parte-Todo Fraca): Ocorre quando os objetos que representam as "partes" mantêm sua existência lógica autônoma mesmo na hipótese de extinção ou destruição do objeto conceitual classificado como "todo". Em termos lógicos, o tempo de vida dos objetos não é atrelado.

Podendo ter como exemplo na fábrica: A relação entre Fornecedor (Todo) e Insumo/Componente (Parte). Caso o cadastro de um fornecedor específico seja excluído do

banco de dados, os componentes por ele fornecidos não perdem a razão de existir e devem persistir registrados no histórico do inventário da fábrica.

Na modelagem física de dados relacionais, implementa-se via FK convencional sem gatilhos de propagação destrutiva (ou aplicando modificações para reter referências nulas / “ON DELETE SET NULL”). No NoSQL MongoDB, opta-se por referências via identificadores lógicos externos (“ObjectId”).

Por outro lado, a composição Estrutural (Relação Parte-Todo Forte) é configurada quando o ciclo de vida existencial dos objetos classificados como "partes" é intrinsecamente dependente e subordinado à existência do objeto "todo". Desfazendo-se o todo, as partes associadas são compulsoriamente eliminadas da memória por consequência lógica.

Cabendo citar um exemplo na Fábrica da relação entre Ordem de Produção - OP (Todo) e os seus respectivos Itens/Apontamentos da Ordem (Parte). Não possui consistência lógica manter registros flutuantes de "itens de ordem" se a Ordem de Produção mãe correspondente for eliminada do sistema operacional. No banco de dados relacional, a composição é garantida fisicamente por meio da restrição de chave estrangeira parametrizada com exclusão em cascata (“ON DELETE CASCADE”). No MongoDB, a composição é representada de forma nativa e ideal por meio de Documentos Embutidos (“Embedded Documents”), nos quais a parte reside fisicamente contida dentro do próprio documento do pai.

2.4.3 Normalização de dados

A normalização é um processo que visa organizar os dados de forma a eliminar redundâncias e evitar inconsistências. Esse processo é guiado pelas chamadas formas normais (SILBERSCHATZ; KORTH; SUDARSHAN, 2020).

Quadro 3 — Formas normais básicas

Forma Normal	Objetivo principal	Problema evitado
1FN	Exige a atomicidade dos campos. Todos os atributos de uma tupla devem conter estritamente valores únicos e indivisíveis, vedando o uso de grupos repetitivos ou vetores multi valorizados.	Campos com múltiplos valores (Ex: Coluna “Telefones” contendo strings separadas por vírgula).

2FN	Exige o cumprimento prévio da 1FN e dita que todos os atributos não-chave devem possuir dependência funcional total em relação à chave primária completa. Só assume relevância prática em tabelas regidas por chaves primárias compostas (duas ou mais colunas).	Dependências Parciais (Ex: Numa tabela com chave composta PedidoID + ProdutoID, o atributo PreçoUnitário depende apenas de ProdutoID, duplicando dados desnecessariamente se o produto figurar em múltiplos pedidos).
3FN	Exige o cumprimento prévio da 2FN e dita que nenhum atributo não-chave pode possuir dependência funcional transitiva em relação a outro atributo não-chave, forçando que todos dependam exclusivamente da chave primária direta.	Dependências Transitivas (Ex: Atributo "NomeGerente" dependendo de" DepartamentoID", que por sua vez depende da chave primária "FuncionarioID". Alterações no gerente exigiriam varreduras em múltiplos registros de funcionários).

Fonte: Adaptado de Silberschatz, Korth e Sudarshan (2020).

A aplicação dessas regras resulta em bancos de dados mais organizados, facilitando tanto a manutenção quanto a análise dos dados.

2.4.4 Relacionamentos entre tabelas

Os relacionamentos estruturam a forma como as entidades se conectam dentro do banco de dados. Os principais tipos são apresentados no Quadro 4.

Quadro 4 — Tipos de relacionamento

Tipo	Descrição	Exemplo em EP
1:1	Um registro relaciona-se com outro único	Máquina ↔ ficha técnica
1:N	Um registro relaciona-se com vários	Fornecedor → pedidos
N:M	Muitos para muitos (tabela intermediária)	Produtos ↔ ordens de produção

Fonte: Adaptado de Elmasri e Navathe (2011).

Esses relacionamentos são essenciais para representar a estrutura real dos sistemas produtivos, permitindo análises integradas entre diferentes áreas.

2.4.5 SQL como ferramenta analítica

A linguagem SQL é o principal meio de interação com bancos de dados relacionais, permitindo realizar consultas, filtragens e agregações de dados (SILBERSCHATZ; KORTH; SUDARSHAN, 2020). Por meio de operações como JOIN e funções de agregação (SUM, AVG, COUNT), é possível gerar indicadores diretamente no banco de dados, reduzindo a dependência de ferramentas externas. No contexto da Engenharia de Produção, isso possibilita análises de: produtividade por linha de produção; níveis de estoque; desempenho de fornecedores e indicadores de qualidade.

2.4.6 MySQL

O MySQL é um SGBD relacional open source desenvolvido originalmente pela empresa sueca MySQL AB e atualmente mantido pela Oracle Corporation. É o banco de dados relacional mais amplamente utilizado no mundo, especialmente em aplicações web, sistemas SaaS e ERPs de médio porte (MYSQL, 2026). Sua arquitetura modular permite a escolha de diferentes engines de armazenamento: o InnoDB, engine padrão desde a versão 5.5, oferece suporte completo a transações ACID, chaves estrangeiras e recuperação automática após falhas, características essenciais para ambientes de produção industrial.

No contexto da Engenharia de Produção, o MySQL é frequentemente encontrado como backend de sistemas MES de pequeno e médio porte, sistemas de gestão de qualidade e plataformas de controle de estoque. Sua facilidade de instalação, ampla documentação e suporte a Python (via mysql-connector-python) e a ferramentas gráficas como MySQL Workbench e DBeaver fazem dele uma escolha natural para o primeiro contato do engenheiro com bancos de dados relacionais.

2.4.7 PostgreSQL

O PostgreSQL é um SGBD relacional open source considerado o mais avançado de sua categoria em conformidade com o padrão SQL e em suporte a tipos de dados complexos (POSTGRESQL, 2026). Originado no projeto POSTGRES da Universidade da Califórnia em Berkeley nos anos 1980, o PostgreSQL destaca-se por recursos que o MySQL não oferece nativamente: tipos de dados avançados como JSON nativo com indexação, arrays, tipos geométricos e GIS (Geographic Information System); suporte a expressões de tabela comuns

(CTEs) recursivas; funções de janela (window functions) para análises temporais; e conformidade estrita com o padrão ANSI SQL.

Para a Engenharia de Produção, o PostgreSQL é a escolha recomendada para sistemas analíticos robustos e para cenários que envolvem dados georreferenciados, como rastreamento de frotas de entrega ou mapeamento de plantas industriais. Sua capacidade de armazenar e indexar documentos JSON sem abrir mão das garantias relacionais o posiciona como uma solução híbrida entre o mundo relacional e o NoSQL. O Quadro 5 sintetiza a comparação entre MySQL e PostgreSQL.

Quadro 5 — MySQL vs. PostgreSQL: Comparativo Técnico

Característica	MySQL	PostgreSQL
Licença	GPL / Oracle	PostgreSQL License (open source)
Conformidade SQL	Moderada	Alta (ANSI SQL)
JSON nativo	Suporte básico	JSONB com indexação avançada
Window Functions	Parcial	Completo
Tipos avançados	Limitado	Arrays, GIS, tipos customizados
Performance leitura	Excelente (simples)	Excelente (consultas complexas)
Uso típico em EP	MES, estoque, web	Analytics, BI, dados geoespaciais

Fonte: elaborado pelos autores com base em MySQL (2026) e PostgreSQL (2026).

2.4.8 MongoDB

O MongoDB é o principal representante dos bancos de dados orientados a documentos. Lançado em 2009 pela empresa 10gen (atual MongoDB Inc.), armazena dados em documentos BSON (Binary JSON), uma representação binária do formato JSON, sem impor um esquema rígido às coleções (MONGODB, 2026). Essa flexibilidade de schema (schema-less) permite que cada documento em uma coleção tenha uma estrutura diferente, o que é particularmente útil em cenários onde os dados variam por tipo de equipamento ou por linha de produção.

No contexto industrial, o MongoDB é frequentemente empregado para o armazenamento de leituras de sensores IoT, onde diferentes equipamentos expõem diferentes conjuntos de

métricas. Um torno CNC pode reportar temperatura do spindle, vibração e corrente elétrica, enquanto uma injetora de plástico reporta pressão de injeção, temperatura do molde e tempo de ciclo, estruturas incompatíveis em um banco relacional sem recorrer a colunas nulas ou tabelas genéricas.

O pipeline de agregação do MongoDB é o equivalente funcional do GROUP BY + JOIN do SQL, permitindo filtrar, desagregar arrays embutidos, agrupar por campos e calcular métricas em uma única operação encadeada. Para análises de séries temporais de sensores, o MongoDB 5.0 introduziu as Time Series Collections, otimizadas para ingestão e compressão de dados temporais com alta frequência.

2.4.9 Redis

O Redis (Remote Dictionary Server) é um banco de dados do tipo chave-valor (key-value store) que opera primariamente em memória RAM, com persistência opcional em disco (REDIS, 2026). Desenvolvido por Salvatore Sanfilippo em 2009, o Redis suporta estruturas de dados ricas além do simples par chave-valor: strings, listas, conjuntos (sets), conjuntos ordenados (sorted sets), hashes e streams, o que o diferencia de outros key-value stores mais simples.

Na Engenharia de Produção, o Redis é aplicado em três cenários principais: (i) cache de consultas, para armazenar em memória os resultados de queries frequentes ao banco relacional, reduzindo o tempo de resposta de dashboards de produção em tempo real; (ii) filas de mensagens, gerenciar a fila de eventos entre sensores IoT e o sistema de processamento, garantindo que nenhuma leitura seja perdida em picos de produção; e (iii) sessões e estado temporário, manter o estado de processos de automação que precisam de acesso ultrarrápido a variáveis intermediárias. O Redis não é adequado como banco de dados primário para dados transacionais, mas como componente de infraestrutura complementar é indispensável em arquiteturas de alto desempenho.

2.4.10 Apache Cassandra

O Apache Cassandra é um banco de dados colunar distribuído, projetado para oferecer alta disponibilidade e escalabilidade linear sem ponto único de falha (CASSANDRA, 2026). Desenvolvido originalmente pelo Facebook em 2008 e doado à Apache Software Foundation, o Cassandra adota uma arquitetura peer-to-peer onde todos os nós são equivalentes, não há nó mestre e os dados são replicados automaticamente entre múltiplos nós e data centers.

A linguagem de consulta do Cassandra, o CQL (Cassandra Query Language), tem sintaxe similar ao SQL, mas com restrições importantes: não suporta JOINS nem subqueries, e o

modelo de dados deve ser projetado orientado às queries o oposto do modelo relacional, onde se modela os dados e depois se define as queries. Essa característica exige um planejamento mais cuidadoso do schema, mas resulta em performance de leitura e escrita previsível e altíssima mesmo com bilhões de registros.

Para a Engenharia de Produção, o Cassandra é o banco ideal para séries temporais de alta frequência: leituras de sensores a cada segundo em centenas de máquinas distribuídas por múltiplas plantas, logs de eventos de máquinas CNC, registros de telemetria de frotas e históricos de alarmes de sistemas SCADA. O Quadro 6 apresenta uma visão consolidada dos quatro SGBDs discutidos.

Quadro 6 — Comparativo Geral dos SGBDs para Engenharia de Produção

SGBD	Tipo	Ponto forte	Limitação principal	Caso de uso em EP
MySQL	Relacional	Simplicidade, ecossistema amplo	Conformidade SQL limitada	MES, estoque, qualidade
PostgreSQL	Relacional	Conformidade SQL, tipos avançados	Configuração mais complexa	Analytics, BI, dados geoespaciais
MongoDB	Documental	Schema flexível, pipeline de agregação	Sem JOINS nativos eficientes	Sensores IoT, catálogos variáveis
Redis	Chave-Valor	Latência sub-milissegundo	Dados limitados pela RAM	Cache, filas, estado em tempo real
Cassandra	Colunar distribuído	Escalabilidade linear, sem falha única	Sem JOINS, modelo query-first	Telemetria, séries temporais massivas

Fonte: elaborado pelos autores (2026).

2.4.11 Abordagem Transacional: Modelos ACID versus BASE sob o Teorema CAP

A estratégia polyglot persistence, termo cunhado por Sadalage e Fowler (2012), preconiza que sistemas complexos devem utilizar múltiplos tipos de banco de dados simultaneamente, cada um aplicado ao problema para o qual foi otimizado, em vez de forçar todos os dados em um único SGBD. Essa abordagem rompe com o paradigma do banco de dados único e universal, reconhecendo que cada tecnologia tem trade-offs e que a escolha deve ser guiada pela natureza dos dados e pelo padrão de acesso.

Em uma planta industrial moderna com arquitetura polyglot, uma configuração típica seria: MySQL ou PostgreSQL para os dados transacionais estruturados do ERP, ordens de produção, notas fiscais, movimentações de estoque, onde as garantias ACID e a integridade referencial são inegociáveis; MongoDB para os registros de sensores IoT com estrutura variável por tipo de equipamento; Redis como cache de dashboard em tempo real e fila de eventos entre o chão de fábrica e o sistema de processamento; e Cassandra para o histórico de séries temporais de longa duração, onde bilhões de leituras precisam ser armazenadas e consultadas com alta performance.

A arquitetura distribuída exige o balanceamento de garantias lógicas transacionais.

Inicialmente pelo Modelo ACID (Garantia Relacional): Adotado por MySQL e PostgreSQL, pauta-se em Atomicidade (a transação ocorre por inteiro ou é abortada), Consistência (o banco migra estritamente de um estado válido a outro), Isolamento (transações simultâneas não geram interferência recíproca) e Durabilidade (os dados persistem gravados mesmo sob colapso de energia). Garante rastreabilidade financeira e patrimonial inegociável em estoques. Em seguida, o Modelo BASE (Garantia NoSQL Distributed): Adotado por bancos como Cassandra e MongoDB, guia-se por Basically Available (o sistema prioriza responder às requisições mesmo com dados momentaneamente dessincronizados), Soft-state (o estado dos dados pode flutuar ao longo do tempo sem intervenção manual) e Eventual consistency (os dados alcançarão a convergência e consistência eventualmente entre todos os nós distribuídos). Prioriza vazão de dados massiva sobre exatidão milimétrica instantânea.

Enquanto o Teorema CAP (provado formalmente por Gilbert e Lynch, 2002) dita a impossibilidade de um sistema de dados distribuído assegurar simultaneamente três propriedades fundamentais: Consistência (*Consistency*), Availability (Disponibilidade) e Partition Tolerance (Tolerância a Partições de Rede).

Havendo uma partição física inevitável na rede industrial (P), caso o sistema escolha Consistência (Sistemas CP - ex: MongoDB), ele rejeitará escritas até que a rede restabeleça-se, sacrificando a disponibilidade momentânea para evitar dados

divergentes. Todavia, se o sistema escolher Disponibilidade (Sistemas AP - ex: Cassandra), ele aceitará as inserções localmente em cada nó, gerando divergências temporárias entre as plantas físicas que serão sanadas via consistência eventual (*BASE*).

O Quadro 7 ilustra essa configuração.

Quadro 7 — Exemplo de Arquitetura Polyglot em Planta Industrial

Categoria do Dado	Volume	SGBD Escolhido	Modelo Transacional	Justificativa de Engenharia de Operações
Ordens e Inventário	Médio	PostgreSQL / MySQL	ACID (Foco em Consistência)	Erros de saldo de estoque ou duplicidade em ordens de produção paralisam a fábrica.
Leituras de Telemetria	Alto	MongoDB	BASE (Foco em Flexibilidade)	Sensores reportam esquemas de dados flutuantes conforme o maquinário.
Métricas em Tempo Real	Baixo	Redis	Memória RAM (Foco em Latência)	Dashboards de andamento de linha exigem atualização instantânea.

Histórico de Logs SCADA	Massivo	Apache Cassandra	BASE (Sistemas AP - Disponibilidade)	Bilhões de telemetrias precisam ser injetadas sem latência ou perda de pacotes.
-------------------------------	---------	---------------------	--	---

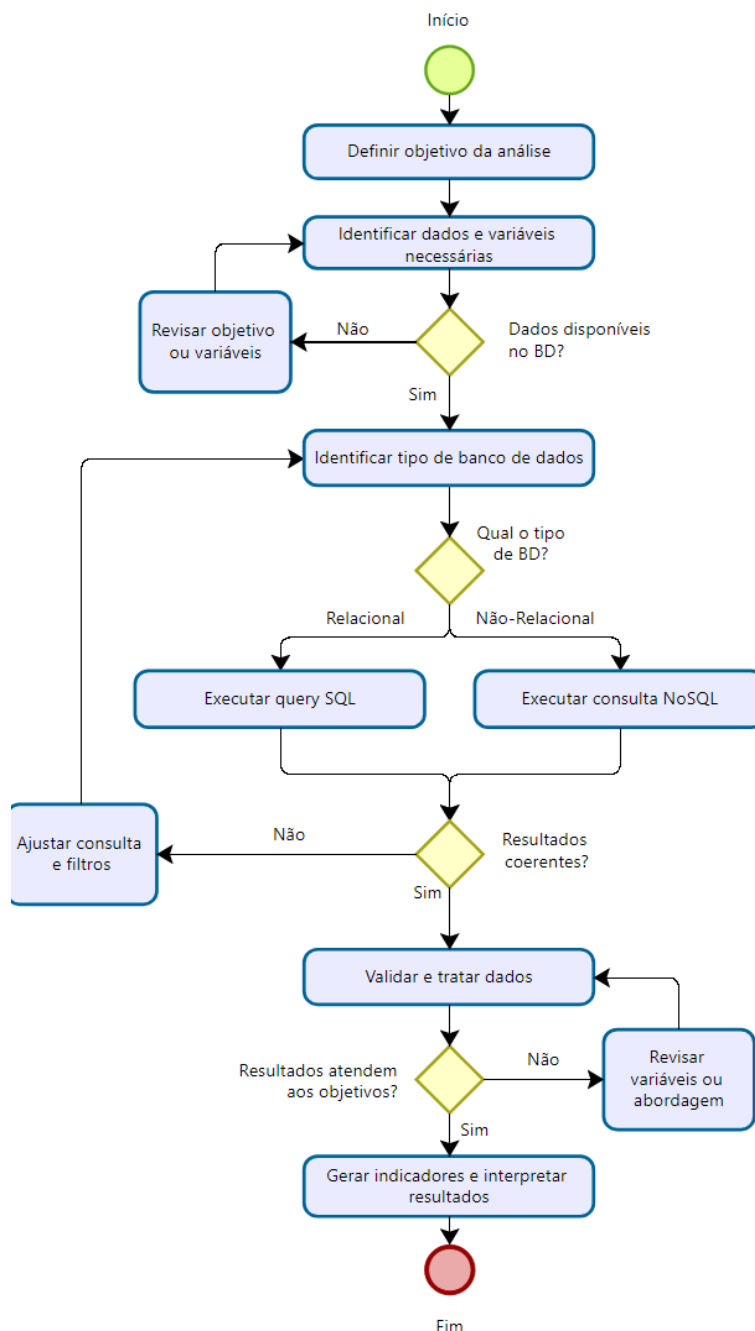
Fonte: elaborado pelos autores com base em Sadalage e Fowler (2012).

3. Metodologia

3.1 Processo de conexão com o banco de dados

A metodologia baseia-se na construção de um Procedimento Operacional Padrão (POP) para acesso e análise de dados em qualquer banco de dados, se limitando, nesse caso, ao MySQL; PostgreSQL; MongoDB e Cassandra, estruturado como um processo iterativo. O fluxo incorpora pontos de decisão que permitem a identificação do banco de dados a ser acessado e o refinamento contínuo da análise, garantindo consistência e aderência aos objetivos propostos.

Figura 1 - Fluxograma do POP de acesso e análise à banco de dados



Fonte: Elaborado pelos autores (2026)

A aplicação do Procedimento Operacional Padrão (POP) para acesso e análise de dados em sistemas de informação segue o fluxo apresentado na Figura 1, no qual cada etapa representa uma fase do processo analítico, incluindo pontos de decisão e ciclos de refinamento.

Inicialmente, realiza-se a definição do objetivo da análise, etapa responsável por direcionar todo o processo. Em seguida, ocorre a identificação dos dados e variáveis necessárias, considerando quais informações são requeridas para atender ao objetivo proposto. Na sequência, verifica-se a disponibilidade dos dados no banco de dados. Caso os dados não

estejam disponíveis, o fluxo retorna à etapa anterior, permitindo a revisão do objetivo ou das variáveis consideradas.

Uma vez confirmada a disponibilidade dos dados, realiza-se a identificação do tipo de banco de dados utilizado pelo sistema de informação. Essa identificação pode ocorrer por meio das configurações de conexão, portas de serviço ou respostas fornecidas pelo próprio servidor, permitindo distinguir bancos relacionais, como MySQL e PostgreSQL, de bancos não relacionais, como MongoDB e Cassandra.

Após essa identificação, procede-se à conexão com o banco de dados e ao mapeamento de sua estrutura. Em bancos relacionais, essa etapa envolve a exploração de tabelas, atributos e relacionamentos por meio de comandos como *SHOW TABLES* e *DESCRIBE*. Já em bancos NoSQL, o processo envolve a identificação de coleções, documentos ou estruturas distribuídas. O entendimento da organização dos dados é essencial para garantir a correta construção das consultas.

Com base nesse mapeamento, inicia-se a etapa de construção e execução das consultas. Em bancos relacionais, utilizam-se comandos SQL como *SELECT*, *WHERE*, *JOIN* e *GROUP BY*, enquanto bancos NoSQL utilizam mecanismos específicos de consulta e agregação. Caso os resultados obtidos não sejam coerentes, o fluxo retorna à própria etapa de construção das consultas, permitindo ajustes em filtros, junções, agregações ou estruturas de busca.

Após a obtenção de resultados consistentes, realiza-se a validação e tratamento dos dados, verificando aspectos como duplicidade, integridade e coerência com o contexto operacional. Na sequência, avalia-se se os resultados atendem ao objetivo da análise. Caso não atendam, o fluxo retorna à etapa de identificação das variáveis ou abordagem analítica, possibilitando o refinamento do processo.

Quando os resultados são satisfatórios, procede-se à geração de indicadores e interpretação dos resultados, etapa em que os dados são transformados em informações úteis para apoio à tomada de decisão. Por fim, todo o processo é documentado na forma de um Procedimento Operacional Padrão (POP), incluindo consultas utilizadas, parâmetros de acesso e finalidade da análise. Essa documentação garante reprodutibilidade, rastreabilidade e padronização das análises futuras.

3.2 Implementação Técnica da Conexão Scriptada via Python

Para operacionalizar o POP com reprodutibilidade técnica estrita e eliminar o viés de extrações manuais, este procedimento estabelece a conexão direta programática utilizando a linguagem Python por meio do driver oficial homologado *mysql-connector-python*.

O bloco de código padrão documentado a ser incorporado nas rotinas analíticas automatizadas do Engenheiro de Produção é detalhado a seguir:

3.3 Construção de Queries SQL e Geração de Indicadores em Engenharia de Produção

Uma vez estabelecida a infraestrutura automatizada de conexão, o POP dita a parametrização das queries lógico-declarativas para extrair dados brutos das tabelas relacionais do chão de fábrica (ordens_producao, paradas_maquina, itens_ordem), convertendo-os diretamente em indicadores de desempenho operacional estruturados.

Para a Geração do Indicador de Qualidade (Taxa de Refugo), o qual mapeia a eficiência de conformidade das linhas de usinagem e injeção plástica.

$$Taxa\ de\ Refugo(\%) = \left(\frac{\Sigma Quantidade\ refugada}{\Sigma Quantidade\ Produzida\ Total} \right) \times 100$$

Enquanto a Geração do indicador de Disponibilidade (Monitoramento de Paradas de Maquinário, o qual quantifica os gargalos temporais físicos por posto de trabalho na fábrica.

4. Resultados Esperados e Planejamento para a fase N2

Como este artigo consolida a etapa conceitual e procedimental de modelagem (Fase N1), os resultados tangíveis estruturados compreendem a capacitação plena do engenheiro de produção para atuar de forma autônoma sobre a base de dados primária.

O planejamento estratégico detalhado para a iminente Fase Prática (N2) deste projeto engloba:

1. Injeção do Dataset de Testes: Carga do esquema físico relacional em ambiente local dockerizado simulando a planta automobilística real com mais de \$100.000\$ tuplas operacionais.
2. Validação dos Indicadores Estruturados: Execução controlada das queries SQL parametrizadas em 3.3 diretamente via scripts Python (pandas), gerando as matrizes numéricas sem intermediação humana.
3. Desenvolvimento de Visualizações Analíticas: Acoplamento dos resultados gerados pelas consultas SQL a ferramentas gráficas open source (Streamlit ou Metabase), consolidando dashboards de OEE em tempo real de forma independente de relatórios estáticos de TI.

5. CONSIDERAÇÕES FINAIS

Este trabalho apresentou os fundamentos teóricos e o procedimento operacional necessários para que o engenheiro de produção acesse, extraia e analise dados diretamente em um banco de dados MySQL. A fundamentação em Sistemas de Informação Gerencial demonstrou que o banco de dados é a camada primária de toda informação organizacional, e que a capacidade

de acessá-lo diretamente representa um diferencial competitivo concreto frente à dependência de relatórios intermediários.

A discussão sobre normalização (1FN, 2FN e 3FN) e tipos de relacionamento evidenciou que um modelo de dados bem estruturado é condição necessária para a confiabilidade das consultas. A comparação entre abordagens relacionais e NoSQL, culminando na estratégia polyglot persistence, posicionou o MySQL como a escolha adequada para dados transacionais estruturados, como ordens de produção, estoques e registros de qualidade, enquanto bancos NoSQL complementam o ecossistema para dados de sensores e telemetria.

Como próximos passos, a parte prática (N2) deste trabalho prevê a conexão a um banco de dados real ou ao dataset disponibilizado pelo professor, com execução das queries documentadas neste POP, geração de indicadores de Engenharia de Produção e análise crítica dos resultados. O objetivo final é demonstrar, com evidências concretas, que o engenheiro de produção que domina dados toma decisões melhores e mais rápidas.

REFERÊNCIAS

DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Campus, 2004.

ELMASRI, R.; NAVATHE, S. B. Sistemas de Banco de Dados. 6. ed. São Paulo: Pearson, 2011.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. Sistema de Banco de Dados. 7. ed. Rio de Janeiro: GEN LTC, 2020.

STAIR, R. M.; REYNOLDS, G. W. Princípios de Sistemas de Informação. 11. ed. São Paulo: Cengage, 2015.

MySQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: mai. 2026.

FOWLER, M. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

MONGODB. MongoDB Manual. Disponível em: <https://www.mongodb.com/docs/manual/>.

Acesso em: mai. 2026.

ABNT - ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR ISO 9001:2015: Sistemas de gestão da qualidade — Requisitos. Rio de Janeiro: ABNT, 2015.

COCKBURN, Alistair. Hexagonal Architecture: Ports and Adapters. Alistair.Cockburn.us, 2005. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>. Acesso em: mai. 2026.

CODD, Edgar F. A relational model of data for large shared data banks. Communications of the ACM, v. 13, n. 6, p. 377-387, 1970.

CORRÊA, Henrique L.; CORRÊA, Carlos A. Administração de Produção e de Operações: manufatura e serviços: uma abordagem estratégica. 4. ed. São Paulo: Atlas, 2017.

DANESHMAND, M.; HARRIS, J. Competing on Analytics: The New Science of Winning. Boston: Harvard Business Review Press, 2007.

DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Campus, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 6. ed. São Paulo: Pearson, 2011.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

GILBERT, Seth; LYNCH, Nancy. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, v. 33, n. 2, p. 51-59, 2002.

LAUDON, Kenneth C.; LAUDON, Jane P. Sistemas de Informação Gerencial. 14. ed. São Paulo: Pearson, 2022.

LAUDON, Kenneth C.; LAUDON, Jane P. Management Information Systems: Managing the Digital Firm. 16. ed. New York: Pearson, 2020.

MARTIN, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017.

MONGODB. MongoDB Manual. Disponível em: <https://www.mongodb.com/docs/manual/>. Acesso em: mai. 2026.

MySQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: mai. 2026.

POSTGRESQL. PostgreSQL 16 Documentation. Disponível em:
<https://www.postgresql.org/docs/>. Acesso em: mai. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2020.

REDIS. Redis Documentation. Disponível em: <https://redis.io/docs/>. Acesso em: mai. 2026.

REENSKAUG, Trygve. Models-Views-Controllers. Xerox PARC Technical Note, 1979.

SADALAGE, Pramod J.; FOWLER, Martin. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston: Addison-Wesley, 2012.

SCHWAB, Klaus. A Quarta Revolução Industrial. São Paulo: Edipro, 2016.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de Banco de Dados. 7. ed. Rio de Janeiro: GEN LTC, 2020.

SLACK, Nigel; BRANDON-JONES, Alistair; JOHNSTON, Robert. Administração de Operações. 8. ed. São Paulo: Atlas, 2018.

STAIR, Ralph M.; REYNOLDS, George W. Princípios de Sistemas de Informação. 11. ed. São Paulo: Cengage, 2015.